

Tarudrishti — Interview Study Guide

Purpose: A deep-dive Q&A reference for explaining the engineering decisions behind Tarudrishti in a technical interview. Each answer is written in first-person, as if you are explaining to a Senior Engineer.

Table of Contents

- 1. [Architecture & Orchestration](#)
- 2. [Backend & Performance](#)
- 3. [Database & Data Model](#)
- 4. [Frontend & UX Engineering](#)
- 5. [Security & Multi-Tenancy](#)
- 6. [DevOps & Deployment](#)

1. Architecture & Orchestration

System Overview

```
graph TB
  subgraph "Frontend – Vercel PWA"
    A["React Client<br/>(Vite + TailwindCSS)"]
  end

  subgraph "Backend – Render"
    B["FastAPI Router"]
    C["LangGraph State Machine"]
    D["APScheduler<br/>(Daily Cron)"]
    E["Mailer Agent"]
  end

  subgraph "External APIs"
    F["OpenAI GPT-4o-mini"]
    G["OpenAI Embeddings"]
    H["Open-Meteo Weather"]
    I["Google OAuth"]
  end

  subgraph "Database – Neon"
    J["PostgreSQL + pgvector"]
  end

  A --> |"JWT + JSON"| B
  B --> C
  C --> |"Structured Output"| F
  C --> |"Embedding"| G
  C --> |"Read/Write"| J
  D --> E
  E --> |"SMTP"| K["Gmail"]
  E --> |"Query"| J
```

```
A -->|"Geolocation"| H
A -->|"OAuth"| I
```

```
A -->|"Geolocation"| H
A -->|"OAuth"| I
```

Q: Why did you use LangGraph instead of a single LLM prompt?

A: A single monolithic prompt would have needed to handle care logging, image diagnosis, and general Q&A all at once. That creates three problems:

1. **Prompt Bloat:** The system prompt would grow to thousands of tokens, increasing latency and cost on every single request — even simple "hello" messages would carry the full logging instruction set.
2. **Output Ambiguity:** A single prompt can't reliably switch between returning structured JSON (for database writes) and free-form markdown (for chat responses). You'd need fragile regex parsing to determine what the LLM "meant."
3. **Error Isolation:** If the image analysis fails, it shouldn't crash the text-based chat. With a monolithic prompt, one failure mode contaminates the entire response.

LangGraph solves this by modeling the conversation as a **compiled state machine**:

```
graph LR
    Entry["🚀 Entry"] --> Router["Router Node<br/>(Intent Classification)"]
    Router -->| "LOG_CARE" | Logger["Logger Agent<br/>(Structured Extraction)"]
    Router -->| "DIAGNOSE_PLANT" | Diag["Diagnostician Agent<br/>(Vision + Text)"]
    Router -->| "GENERAL_CHAT" | Bot["Botanist Agent<br/>(pgvector Cache)"]
    Router -->| "Unknown" | Err["Error Node<br/>(Graceful Fallback)"]
    Logger --> End1["✅ END"]
    Diag --> End2[End]
    Bot --> End3[End]
    Err --> End4[End]
```

The Router Node uses **OpenAI Structured Output** with a Pydantic model (`IntentClassification`) to classify intent with a confidence score. This isn't regex or keyword matching — it's a constrained decoder that guarantees valid enum output.

Each specialist has its own system prompt, its own output schema, and its own error handling. The Logger Agent returns `CareLogExtraction` (a list of `SingleCareAction` objects). The Diagnostician returns free-form markdown. They never interfere with each other.

Q: How does `graph.py` maintain state across nodes?

A: LangGraph uses a `TypedDict` called `AgentState` that flows through the graph:

```
class AgentState(TypedDict):
    messages: List[AnyMessage] # Conversation history
    current_date: str # Temporal context for the Logger
    image_base64: Optional[str] # Multimodal input for Diagnostician
    intent: Optional[IntentType] # Set by Router, read by route_intent
    final_response: Optional[Dict] # Set by specialist, read by FastAPI
    plants_context: List[Dict] # User's plants for grounding
    db: Any # SQLAlchemy session for cache/writes
```

Each node receives the full state, performs its computation, and returns a **partial update dictionary**. LangGraph merges the update automatically. For example, the Router Node only returns `{"intent": response.intent}` — it doesn't touch `messages` or `db`. This is a functional programming pattern — pure input/output with no side effects between nodes.

Q: What happens if the Router Node hallucinates an invalid intent?

A: Two layers of defense:

```
graph TD
    A["Router invokes<br/>llm.with_structured_output()"] -->| "Success" | B["Returns valid IntentType enum"]
    A -->| "Exception<br/>(parse error, network)" | C["Catches exception"]
    C --> D["Returns GENERAL_CHAT<br/>as safe default"]
    B --> E["route_intent() checks enum"]
    E -->| "Known intent" | F["Routes to specialist"]
    E -->| "Unknown value" | G["Routes to error_node<br/>(graceful message)"]
```

1. **Structured Output Constraint:** `llm.with_structured_output(IntentClassification)` constrains the LLM's decoder to only output valid `IntentType` enum values. Invalid tokens are near-impossible.
2. **Deterministic Fallback:** If parsing throws an exception, the Router catches it and returns `GENERAL_CHAT`. If `route_intent` receives an unrecognized enum, it routes to a dedicated `error_node`. There is **no code path** that can produce an unhandled edge.

2. Backend & Performance

Q: Explain the Semantic Cache. Why `pgvector` + `text-embedding-3-small` ?

A: The Botanist Agent answers general questions like "How often should I water a snake plant?" These are asked repeatedly by different users with minor phrasing variations.

```
graph TD
    A["User asks:<br/>'How to water snake plant?'" ] --> B["Generate 1536-dim embedding<br/>via text-embedding-3-small"]
    B --> C{"cosine_distance < 0.15?<br/>(85%+ similarity)"}
    C -->| "Cache HIT" | D["Return cached response<br/>↗ Zero LLM call"]
    C -->| "Cache MISS" | E["Call GPT-4o-mini"]
    E --> F["Store response + embedding<br/>in semantic_cache table"]
    F --> G["Return fresh response"]
```

```
db.query(SemanticCache).filter(
    SemanticCache.embedding.cosine_distance(query_embedding) < 0.15
).first()
```

Why not a local HuggingFace model? The backend runs on Render's Starter tier with **512MB RAM**.

Loading even a small sentence-transformer (`all-MiniLM-L6-v2` at ~90MB) would consume 20%+ of memory. With 4 Gunicorn workers, that's 360MB just for embeddings — OOM under load.

`text-embedding-3-small` is a **stateless API call** — zero memory footprint on the server. Cost: \$0.00002 per 1K tokens, negligible vs. the LLM inference it saves.

Why 0.15 threshold? Empirically tuned. At 0.05 (original), "How do I water a snake plant?" wouldn't match "How frequently should snake plants be watered?" At 0.15, semantically equivalent phrasings hit the cache while genuinely different questions correctly miss.

Q: Why `pool_size=3`, `max_overflow=5` in SQLAlchemy?

A: Each Unicorn worker gets its own connection pool:

```
Max Connections = workers × (pool_size + max_overflow)
                 = 4 × (3 + 5) = 32
```

Neon's free tier allows **100 concurrent connections**. SQLAlchemy defaults (`pool_size=5`, `max_overflow=10`) would yield $4 \times 15 = 60$ — dangerously close when factoring in the APScheduler thread and migration connections.

- `pool_recycle=300` — Recycles connections every 5 min to prevent Neon's serverless proxy from killing idle ones.
- `pool_pre_ping=True` — Sends `SELECT 1` before each checkout to verify the connection is alive.

Q: How does the APScheduler email notification system work?

A: `scheduler.py` starts a `BackgroundScheduler` (in-process thread) during FastAPI's lifespan startup:

```
graph LR
    A["FastAPI Startup"] --> B["APScheduler starts"]
    B -->|"CronTrigger<br/>daily 8:00 AM"| C["check_and_send_daily_notifications()"]
    C --> D["Query all plants + care_logs"]
    D --> E["Compute overdue tasks<br/>Water: 7-day interval<br/>Fertilize: 30-day interval"]
    E --> F["Group tasks by user email"]
    F --> G["Render HTML email per user"]
    G -->|"SMTP configured"| H["Send via Gmail"]
    G -->|"No SMTP"| I["Mock to console"]
```

If SMTP credentials aren't configured, it gracefully falls back to console output — no crash, no silent failure.

3. Database & Data Model

Q: Walk me through the database schema.

A: Four tables with clear ownership chains:

```
erDiagram
    USERS {
        int id PK
        string email UK
        string hashed_password
    }
```

```

PLANTS {
  int id PK
  string name
  string species
  string health_status
  string image_url
  int user_id FK
  datetime created_at
}
CARE_LOGS {
  int id PK
  int plant_id FK
  string action_type
  string substance_used
  datetime action_date
  datetime created_at
}
SEMANTIC_CACHE {
  int id PK
  string query_text
  string response_text
  vector1536 embedding
  datetime created_at
}

USERS ||--o{ PLANTS : "owns (cascade delete)"
PLANTS ||--o{ CARE_LOGS : "has (cascade delete)"

```

Key design decisions:

- **Cascade deletes:** `cascade="all, delete-orphan"` on relationships means deleting a user automatically deletes all their plants and care logs. No orphaned data.
- **image_url stores base64:** The compressed image data URL is stored directly in Postgres, avoiding an external object storage dependency (S3, Cloudinary) for an MVP.
- **SemanticCache is tenant-agnostic:** General botanical answers are shared across all users — "How to water a snake plant?" has the same answer regardless of who asks.
- **Vector(1536)** : The pgvector column stores OpenAI `text-embedding-3-small` embeddings for cosine distance queries.

Q: How does the ScheduleView compute overdue tasks?

A: The `ScheduleView` and `PlantProfile` both use interval-based computation against care logs:

```

// For each plant, find the most recent "Water" log
const waterLogs = plant.care_logs.filter(log => log.action_type.includes('water'));
waterLogs.sort((a, b) => new Date(b.action_date) - new Date(a.action_date));
const lastWatered = waterLogs[0]?.action_date;

// Compute days since last watering
const diffDays = (now - lastWatered) / (1000 * 60 * 60 * 24);

```

```
// If >= 7 days have passed, it's overdue
const isOverdue = diffDays >= 7;
```

This runs **entirely on the frontend** using data already fetched by React Query. No extra API call needed. The intervals are: Water (7 days), Fertilize (21 days), Pesticide (30 days).

4. Frontend & UX Engineering

Q: Walk me through the React component tree.

```
graph TD
    Root["main.jsx"] --> Providers["GoogleAuthProvider<br/>QueryClientProvider<br/>ThemeProvider<br/>AuthProvider"]
    Providers --> App["App.jsx"]
    App -->|"No user"| Landing["LandingPage<br/>(Auth forms + Google OAuth)"]
    App -->|"User + plant selected"| Profile["PlantProfile<br/>(Dashboard + Gallery tabs)"]
    App -->|"User authenticated"| Layout["Layout<br/>(Header + BottomNav)"]
    Layout -->|"gallery tab"| Gallery["PlantGallery"]
    Layout -->|"schedule tab"| Schedule["ScheduleView"]
    Layout --> Chat["AIChatSheet<br/>(Bottom sheet + AI)"]
    Gallery --> Card["PlantCard"]
    Gallery --> Weather["WeatherWidget"]
    Gallery --> AddModal["AddPlantModal"]
```

Key patterns:

- **React Query** manages all server state. When the AI logs a care activity, it calls `queryClient.invalidateQueries({ queryKey: ['plants'] })` — the gallery and schedule re-fetch automatically.
- **No React Router:** Navigation is managed via `useState` (`activeTab` , `selectedPlant`). The app is a single-screen PWA, not a multi-page SPA.
- **Context providers** are layered: Theme → Auth → OnlineStatus. Each is independent and doesn't re-render siblings.

Q: How does the client-side image compression work?

A: When a user takes a photo on a modern phone, the raw file is 3-8MB. After base64 encoding, that's ~4-11MB in the JSON payload.

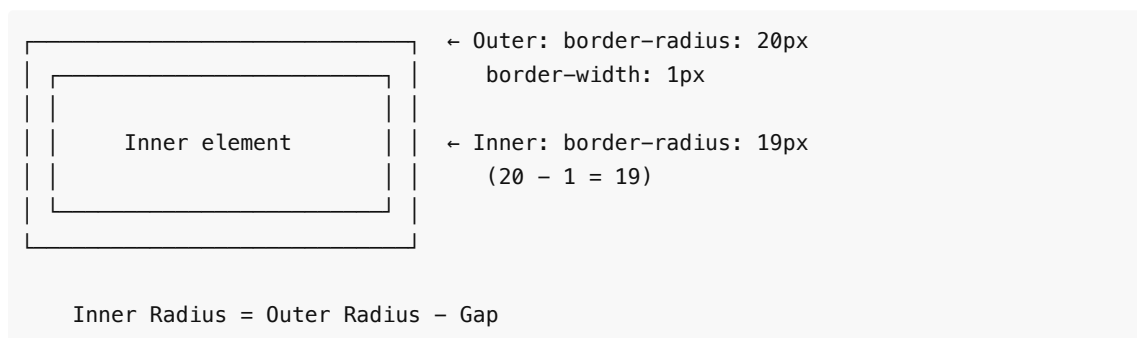
```
graph LR
    A["📷 Phone Camera<br/>4000x3000 PNG<br/>~5MB"] --> B["FileReader<br/>readAsDataURL()"]
    B --> C["new Image()"]
    C --> D["Calculate ratio<br/>Max 1024px"]
    D --> E["Canvas.drawImage()<br/>at reduced size"]
    E --> F["canvas.toDataURL<br/>('image/jpeg', 0.8)"]
    F --> G["📄 1024x768 JPEG<br/>~150KB base64"]
```

```
const MAX = 1024;
const ratio = Math.min(MAX / width, MAX / height);
width = Math.round(width * ratio); // Maintains aspect ratio
height = Math.round(height * ratio);
const canvas = document.createElement('canvas');
ctx.drawImage(img, 0, 0, width, height);
canvas.toDataURL('image/jpeg', 0.8); // 80% quality JPEG
```

Why this matters: Render's proxy times out at 30s. A 10MB payload on a 1Mbps mobile connection takes ~80 seconds. After compression: ~150KB uploads in under 2 seconds. Zero external libraries — hardware-accelerated Canvas API only.

Q: Explain the nested border-radius rule.

A: When you nest two rounded rectangles, the inner element's radius must account for the gap between them:



In `PlantCard.jsx` :

```
<motion.div className="rounded-[20px]" style={{ border: '1px solid ...' }}>
  <div className="rounded-[19px]"> /* 20 - 1 = 19 */
```

If both had `20px`, the curves wouldn't track — the inner curve overshoots at corners, creating visible gaps where content "bleeds." Apple's iOS follows this rule on every nested element.

Q: How do the Framer Motion spring physics work?

A: Instead of CSS `transition: all 0.3s ease`, the app uses physics-based springs:

```
{ type: "spring", stiffness: 400, damping: 30, mass: 1 }
```

Why springs over CSS transitions?

1. **Interruptibility:** If a user taps during animation, a spring redirects from current velocity. CSS transitions snap.
2. **Velocity-awareness:** `whileTap={{ scale: 0.95 }}` releases with natural momentum — organic "bounce back."
3. **Consistency:** Centralized `springConfig` in `animations.js` ensures every element moves with identical physics.

Values `stiffness: 400`, `damping: 30` mimic iOS's default spring curve — responsive but not bouncy.

Q: How does the voice input work?

A: The `AIChatSheet` uses the **Web Speech API** (`webkitSpeechRecognition`):

```
const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;
const recognition = new SpeechRecognition();
recognition.continuous = true;
recognition.interimResults = true; // Live transcription as user speaks

recognition.onresult = (event) => {
  // Accumulate transcript and set it as input text
  setInputText(prefix + transcript);
};
```

The UI swaps the input pill for a `RecordingPill` with an animated waveform (24 bars with randomized CSS animation durations). This allows fully hands-free interaction while gardening.

Q: How does the weather context reach the AI?

A: Three-step pipeline:

1. **WeatherWidget** requests browser geolocation → calls Open-Meteo API → stores result in `localStorage` under `tarudrishti_weather_context`.
2. **AIChatSheet** reads from `localStorage` before each message and injects it into the `ChatRequest` payload as `weather_context`.
3. The LLM system prompt receives it as contextual grounding: *"Current Location: Delhi, Temperature: 42°C"* → AI responds: *"At 42°C, water your snake plant every 3 days instead of weekly."*

Q: What is the Typewriter effect?

A: New AI responses render character-by-character using a `Typewriter` component:

```
function Typewriter({ text, speed = 15 }) {
  const [displayedText, setDisplayedText] = useState('');
  const [index, setIndex] = useState(0);

  useEffect(() => {
    if (index < text.length) {
      const timeout = setTimeout(() => {
        setDisplayedText(prev => prev + text[index]);
        setIndex(prev => prev + 1);
      }, speed); // 15ms per character
      return () => clearTimeout(timeout);
    }
  }, [index]);

  return <ReactMarkdown>{displayedText}</ReactMarkdown>;
}
```

Only the **newest** message gets the typewriter effect (via an `isStreaming` flag). Older messages in history render instantly. This prevents re-typing the entire conversation on each re-render.

5. Security & Multi-Tenancy

Q: How does `get_current_user` prevent tenant data leakage?

```
graph TD
    A["HTTP Request<br/>Authorization: Bearer eyJhb..."] --> B["oauth2_scheme<br/>extracts raw JWT"]
    B --> C["jwt.decode(token, SECRET_KEY)<br/>validates HMAC-SHA256 signature"]
    C --> D["Invalid/Expired"]
    C --> E["Valid"]
    D --> F["401 Unauthorized"]
    E --> G["Extract email from 'sub' claim"]
    G --> H["Missing"]
    G --> I["Present"]
    H --> J["D"]
    I --> K["db.query(User).filter(email)"]
    K --> L["Not found"]
    K --> M["Found"]
    L --> N["D"]
    M --> O["Returns User object<br/>with user.id"]
    O --> P["Every query filters by user.id"]
```

Every protected endpoint receives `current_user: models.User = Depends(auth.get_current_user)`. Every database query then filters:

```
db.query(Plant).filter(Plant.user_id == current_user.id)
```

Why is this "mathematically" secure?

- The `user_id` comes from `jwt.decode()` — a **cryptographic verification**, not user input. The user cannot modify their token's `sub` claim without invalidating the HMAC-SHA256 signature.
- The `SECRET_KEY` is the only value that can produce a valid signature. In production, it's auto-generated by Render and never exposed.
- There is **no endpoint** that queries `Plant` or `CareLog` without the `user_id` filter.

Therefore: User A's token → User A's `user.id` → only User A's rows. No injection point, no IDOR vector.

Q: How does Google OAuth work in this system?

```
sequenceDiagram
    participant U as User Browser
    participant G as Google OAuth
    participant F as Frontend
    participant B as Backend

    U->>G: Click "Sign in with Google"
    G-->>F: Returns Google JWT credential
    F->>F: jwtDecode(credential) → extract email
    F->>B: POST /api/auth/google { email }
    B->>B: Find or create User by email
    B->>B: Generate backend JWT with user.email as "sub"
    B-->>F: { access_token, token_type }
    F->>F: Store { email, name, token } in localStorage
    F->>F: AuthContext.login() → re-renders App
```

The backend **never sees the Google token**. It only receives the email, finds or creates a user with a random unguessable password, and issues its own JWT. This means the JWT authority is always the backend's `SECRET_KEY`, regardless of auth method.

Q: Why 60-minute token expiration instead of 7 days?

A: A 7-day token is a liability. If compromised (stolen phone, XSS), the attacker has a week of access. With 60 minutes, the blast radius is limited to 1 hour. In a full production system, you'd add a **refresh token** flow (HttpOnly cookie) for seamless re-authentication.

Q: Why remove the CORS origin regex?

A: The original `allow_origin_regex="https://tarudrishti-.*\\.vercel\\.app"` matches any subdomain like `tarudrishti-malicious-site.vercel.app`. Since anyone can deploy to Vercel, an attacker could register that domain and make authenticated cross-origin requests. Replaced with an explicit allowlist.

6. DevOps & Deployment

Q: Why Gunicorn wrapping Uvicorn workers?

```
graph TD
    A["Gunicorn Master Process<br/>(no application code)"] --> W1["Uvicorn Worker 1"]
    A --> W2["Uvicorn Worker 2"]
    A --> W3["Uvicorn Worker 3"]
    A --> W4["Uvicorn Worker 4"]

    W2 -->|"💥 Crash (OOM)"| A
    A -->|"Fork new worker<br/>~1 second"| W5["Uvicorn Worker 5<br/>(replacement)"]
```

Raw `uvicorn main:app` runs a single process. If it crashes, the entire service goes down for ~30-60 seconds until Render restarts it. With Gunicorn:

- 4 workers serve requests concurrently.
- If one crashes, the other 3 continue with zero downtime.
- Gunicorn forks a replacement worker in ~1 second.

Q: Walk me through the cold-start interceptor.

A: Render spins down after 15 minutes of inactivity. Cold start takes 15-30 seconds.

```
let coldStartToast;
const coldStartTimeout = setTimeout(() => {
  coldStartToast = toast.loading('Waking up server... up to 30s.');
```

```
}, 3000); // Only triggers if response takes > 3 seconds

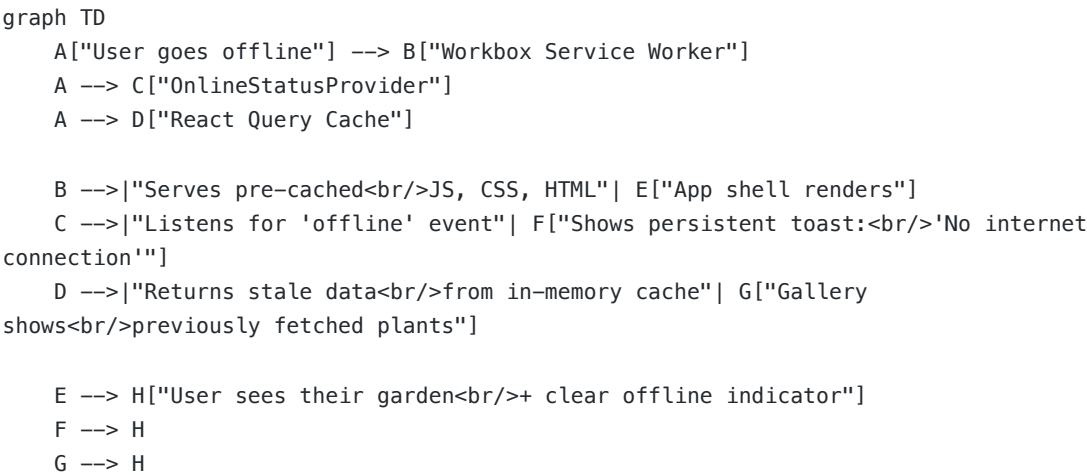
try {
  const response = await fetch(url);
  clearTimeout(coldStartTimeout);
  if (coldStartToast) toast.dismiss(coldStartToast);
```

```
    return response;
} catch (error) {
  clearTimeout(coldStartTimeout);
  if (coldStartToast) toast.dismiss(coldStartToast);
  throw error;
}
```

The 3-second threshold is intentional: warm responses take 200-800ms. If 3 seconds pass, it's almost certainly a cold start, and the user gets honest feedback instead of a frozen UI.

Q: How does the PWA offline behavior work?

A: Three layers:



If a user walks into their garden and loses WiFi, they see their plants, their schedule, and a clear "Offline Mode" indicator — not Chrome's dinosaur.

Quick Reference: Key File Map

File	Purpose
backend/graph.py	LangGraph state machine — Router, Logger, Diagnostician, Botanist, Error nodes
backend/auth.py	JWT creation, verification, get_current_user dependency
backend/main.py	FastAPI app, CORS, REST endpoints, orchestrator endpoint
backend/models.py	SQLAlchemy models: User, Plant, CareLog, SemanticCache (pgvector)
backend/schemas.py	Pydantic schemas: IntentClassification, CareLogExtraction, ChatRequest
backend/database.py	SQLAlchemy engine, connection pooling, pgvector extension init

backend/mailer.py	Email notification agent with overdue task computation
backend/scheduler.py	APScheduler cron trigger for daily mailer
src/App.jsx	Root component — conditional rendering based on auth and navigation state
src/utils/api.js	apiFetch wrapper with JWT injection and cold-start interceptor
src/utils/animations.js	Centralized Framer Motion spring physics constants
src/components/AIChatSheet.jsx	Chat UI, image compression, voice input, typewriter effect
src/components/PlantCard.jsx	Gallery card with nested border-radius and health indicators
src/components/PlantProfile.jsx	Plant detail view with care metrics and activity timeline
src/components/ScheduleView.jsx	Care schedule with interval-based overdue computation
src/components/WeatherWidget.jsx	Geolocation + Open-Meteo weather with AI care suggestions
src/components/LandingPage.jsx	Auth forms, Google OAuth, animated SVG botanicals
src/components/OnlineStatusProvider.jsx	Browser online/offline event listener with toast
src/context/AuthContext.jsx	JWT persistence in localStorage, login/logout state
src/context/ThemeContext.jsx	Dark/light mode via CSS custom properties + data-theme

Interview Framing Tip: Lead with the **problem** before the solution. Don't say "I used LangGraph." Say "I needed to route user intents to specialized agents with different output schemas, so I modeled it as a state machine using LangGraph." That framing demonstrates engineering judgment, not just tool familiarity.